Bansilal Ramnath Agarwal Charitable Trust's

Vishwakarma Institute of Information Technology, Pune - 411048

(An Autonomous Institute affiliated to Savitribai Phule Pune University)

**Department of CSE- Data Science 2026-27 Sem-I**

**B.TECH. MAJOR PROJECT SYNOPSIS**

**1. Title of the Project**

**AegisRAG: A Self-Verifying Agentic Email Assistant with RAG-Based Prioritization, Phishing Detection, and Claim-Level Explanation Verification**

**2. Student Details**

| Sr. No. | Name of Student | PRN | Roll No. |
|---|---|---|---|
| 1 | Tanishq Raina | 22310084 | 17 |
| 2 | Alpesh Thool | 22310727 | 05 |
| 3 | Tayyab Mohammed | 22311292 | 32 |
| **4** | Chinmay Kale | 22310757 | 21 |

**3. Project Guide**

**Name**: Mandar Diwakar

**4. Synopsis / Project Description**

## 4.1 Problem Statement:

Existing agentic email-management systems, including the author's own MailPilot system, use Retrieval-Augmented Generation (RAG) to prioritize messages and, increasingly, to explain their decisions in natural language, but leave two related risks unaddressed: first, prioritization is treated as unrelated to security, a phishing email written to appear urgent or personally relevant can be prioritized by such a system over being flagged, and static blacklist-based filters fail against novel, LLM-generated phishing content; second, and more fundamentally, the natural-language explanations that agentic systems generate for their own decisions are not verified against the evidence they were retrieved from, a plausible-sounding but ungrounded justification can accompany an incorrect or even correct decision without ever being checked. As agentic AI systems are given more autonomy to flag, quarantine, or act on email automatically, an unverified explanation is not just a cosmetic issue but a trust and safety gap. There is currently no lightweight, self-hostable agentic email system that unifies RAG-based prioritization, real-time phishing detection, and claim-level verification of its own explanations in a single pipeline

## 4.2 Abstract:

MailPilot is an existing agentic AI system that uses Retrieval-Augmented Generation (RAG) and rule-based classification to prioritize email, but, like most agentic assistants, treats security as a separate concern and does not verify the natural-language explanations it generates for its own decisions. This project proposes MailPilot-Verify, which extends MailPilot along two complementary directions in a single unified pipeline: first, a phishing and social-engineering detection module is added alongside the

existing priority classifier, using sender-authentication checks (SPF/DKIM/DMARC), URL/domain reputation analysis, and LLM-based semantic pattern detection, fused into a single explainable risk score; second, every explanation the system generates, whether justifying a priority ranking or a phishing flag, is decomposed into atomic claims and checked against the retrieved evidence using a Natural Language Inference (NLI) model, so that ungrounded or hallucinated reasoning is flagged or removed before it reaches the user. The system is evaluated in two phases: detection performance (accuracy, false-positive rate, latency) against public phishing-email benchmarks, and explanation quality (reduction in ungrounded claims, verifier precision/recall) against a constructed test set that includes deliberately unanswerable 'trap' cases. The expected outcome is a single, self-hostable agentic email assistant that is simultaneously more secure and more trustworthy, closing both the phishing-detection gap and the explanation-grounding gap in one coherent system rather than two disconnected tools.s.

### 4.3 Keywords

Agentic AI; Retrieval-Augmented Generation (RAG); Large Language Models (LLM); Phishing Detection; Email Security; Hallucination Detection; Natural Language Inference (NLI); Claim Verification; Explainable AI (XAI); Sender Authentication (SPF/DKIM/DMARC); Trustworthy AI; FastAPI; Gmail API

## 4.4. Proposed Methodology / Technology:

Recent work has demonstrated that large language models are capable of both generating convincing phishing content and, when appropriately prompted, detecting its intent [1], [2], [3], often matching or outperforming rule-based filters against new attacks. Heiding et al. [1] have demonstrated that LLM-generated phishing emails achieve click-through and credibility rates comparable to human-crafted attacks, which suggests that the same generative capability can be applied to recognizing the linguistic and structural cues of social engineering in order to detect phishing attempts beyond static indicators such as URLs and signatures known at the time of writing a rule-based classifier. Koide et al.'s ChatSpamDetector [2], and Chataut et al. [3], independently confirm that LLM-based classifiers generalize better than traditional filters against novel phishing campaigns that have not had a known signature to match against, which is the primary vulnerability in MailPilot's existing rule-based classification system.

Separately, hallucination benchmarks such as RAGTruth [11] have demonstrated that even retrieval-grounded systems frequently generate unsupported claims. Niu et al. have found that hallucination rates in RAG pipelines remain substantial even when the generator has direct access to correct source documents, suggesting that simply grounding a model in a set of documents is not sufficient to ensure grounded output. This has implications for an agentic system such as MailPilot-Verify, in which retrieving the correct context is necessary but not sufficient to ensure trustworthiness, since the language model can still assert a claim that is supported by evidence that is not present. Fact-checking approaches that verify claims directly against grounding documents [16] have proven effective in catching these errors after the fact, and Tang et al.'s MiniCheck has demonstrated that lightweight, purpose-built fact-checking models can verify LLM output against source documents with accuracy comparable to much larger LLM-as-judge approaches at a fraction of the inference cost, making it practical to run a claim-level verification on every email instead of performing it only once per rule-based classifier.

Bansilal Ramnath Agarwal Charitable Trust's

Vishwakarma Institute of Information Technology, Pune - 411048

(An Autonomous Institute affiliated to Savitribai Phule Pune University)

**Department of CSE- Data Science 2026-27 Sem-I**

MailPilot-Verify combines both lines of work into a single pipeline: MailPilot's existing RAG-based retrieval is extended with a parallel phishing-detection path, and every explanation produced by either is passed through a claim-level verifier before being given to the user rather than treating detection and explanation-trustworthiness as separate projects. This approach is a deliberate choice rather than a convenient merger: a system that is able to better detect phishing but generates unverified, potentially hallucinated justifications for its decisions has only solved half the trust problem, since a user who cannot trust the system's stated reasoning is unlikely to trust its automated actions either.

1. Email Ingestion: Emails are pulled in real time via the Gmail API and normalized into a common format (headers, body text, links, attachment metadata), reusing MailPilot's existing ingestion layer.
2. Preprocessing & Feature Extraction: Text is cleaned and key features (URLs, sender domain, display-name mismatches, header anomalies) are extracted for downstream analysis.
3. Sender Authentication: SPF, DKIM, and DMARC DNS records are verified in real time to detect spoofed sending domains, since a failed authentication check is a strong, cheap-to-compute early signal.
4. RAG Retrieval Core: MailPilot's existing retrieval mechanism fetches similar past emails and relevant context, which feeds both the priority classifier and the new phishing detector so neither path works from a different or inconsistent context.
5. Dual-Path Analysis: The priority classifier computes a relevance score and a natural-language justification, while the phishing detector analyzes URL/domain reputation and LLM-identified social-engineering patterns (urgency, authority impersonation, credential requests) to produce a risk score and its own justification.
6. Explanation Generation: The two justifications are combined into a single, coherent natural-language explanation describing why the email was ranked and/or flagged the way it was.
7. Claim Decomposition & Verification: The combined explanation is split into atomic claims, and each claim is checked against the retrieved context and extracted features using an NLI model (e.g., a RoBERTa-based entailment classifier), labeling it entailed, contradicted, or unsupported.
8. Risk Fusion, Filtering & Dashboard: Unsupported or contradicted claims are flagged or removed from the explanation, the priority and phishing scores are fused into a single risk output, and the user sees a verified, evidence-linked explanation in a dashboard, which drives the flag/quarantine/prioritize action.

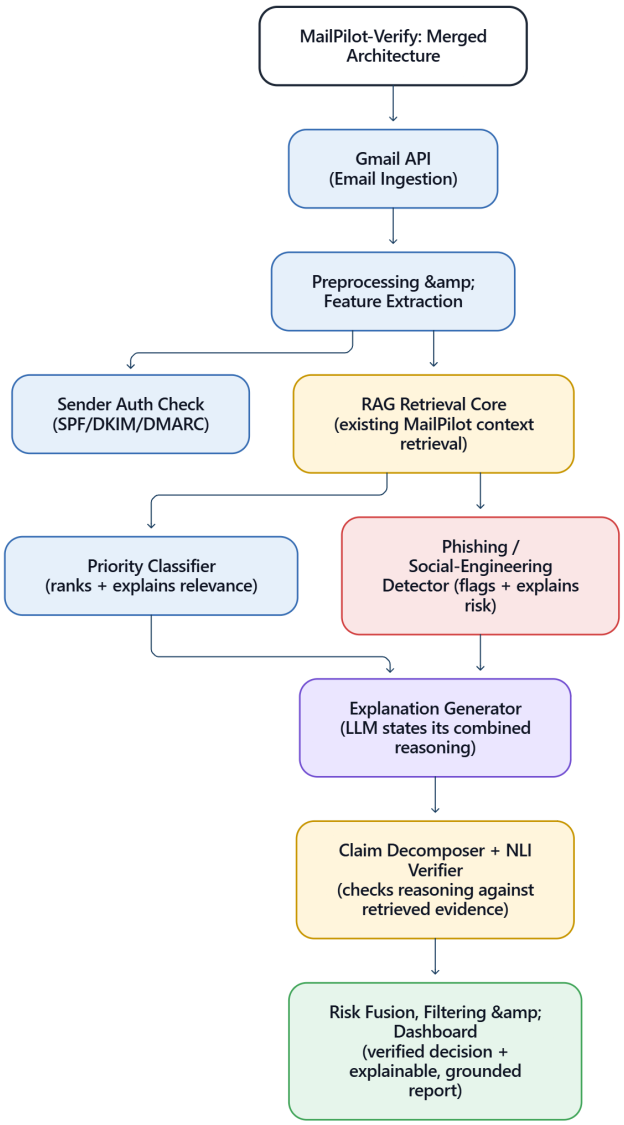



*Fig. 1 — Merged MailPilot-Verify architecture: RAG-based retrieval feeds a dual-path priority/phishing analysis, whose combined explanation is claim-verified before reaching the user.*

Evaluation Approach: Because this project combines two previously separate contributions, it is evaluated in two phases rather than one. Phase 1 measures detection performance in isolation, using a held-out split of a labeled phishing/legitimate email dataset to report accuracy, false-positive rate, and per-email latency. Phase 2 measures explanation quality: the same query set is run once without the claim verifier and once with it, including deliberately constructed 'trap' cases designed to expose ungrounded reasoning, to measure both the reduction in ungrounded claims and the verifier's own precision/recall. Fig. 2 summarizes both phases.

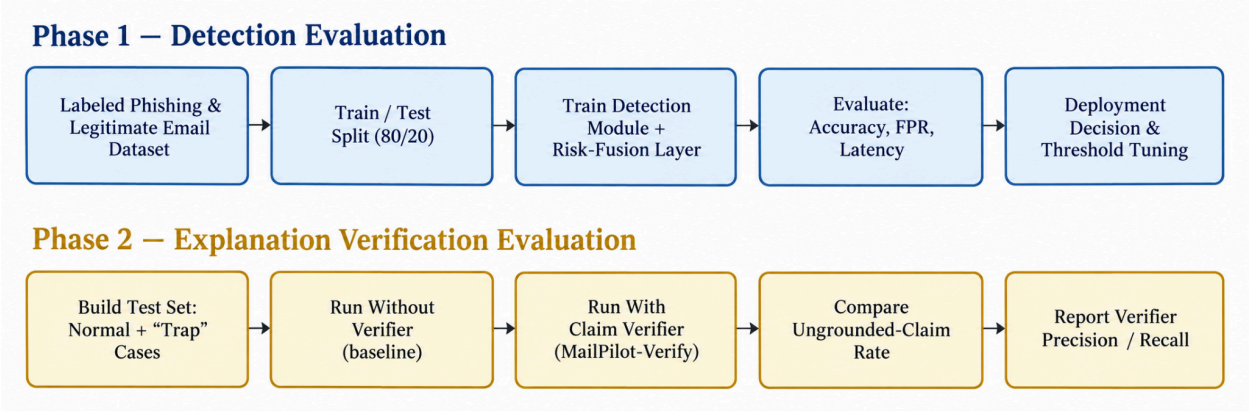



*Fig. 2 — Two-phase evaluation plan: detection performance (Phase 1) and explanation-verification performance (Phase 2), evaluated independently.*

### 4.5 Tools / Technologies

| Category | Tool / Technology | Purpose |
|---|---|---|
| Backend / API | Python, FastAPI | Core service layer extending MailPilot's existing backend |
| Agentic / RAG Framework | LangChain | Orchestrates retrieval, dual-path analysis, and explanation generation |
| Email Ingestion | Gmail API | Real-time email retrieval, reusing MailPilot's existing OAuth-scoped ingestion |
| LLM | Anthropic/OpenAI API or open-source LLM (e.g., Llama 3) | Priority scoring, phishing pattern detection, and explanation generation |
| Vector Database | FAISS / Chroma | Stores embeddings for MailPilot's existing RAG-based retrieval |
| Sender Authentication | dnspython | SPF/DKIM/DMARC DNS record verification |
| NLI Verification Model | HuggingFace Transformers (e.g., roberta-large-mnli) | Classifies each explanation claim as entailed, contradicted, or unsupported |
| Classical ML | scikit-learn | Supplementary URL/domain reputation and feature-based classifiers |
| Storage | PostgreSQL / SQLite | Logging flagged emails, risk scores, explanations, and |

| | | |
|---|---|---|
| | | evaluation results |
| Deployment | Docker | Containerized, self-hostable deployment |
| Dashboard (optional) | React / Next.js | User-facing risk dashboard with verified, evidence-linked explanations |

**4.6 expected outcome**

A single deployed extension of MailPilot that is simultaneously more secure and more trustworthy. For Phase 1, we target a detection accuracy on public phishing-email benchmarks exceeding 90%, a false positive rate under 5%, less than 2 seconds per email processing latency, and no significant drop in MailPilot's current prioritization accuracy after integration. For Phase 2, we target at least a 30-40% relative decrease in ungrounded/contradicted explanation claims made to the user (relative to some unverified baseline) and a verifier with precision exceeding 85% so that correct explanations are rarely wrongly flagged. The final deliverable will be a dashboard showing, for each email, a fused risk score along with a verified, evidence-linked explanation of why that decision was made.

.

**4.7 Value Addition / Innovation**

1. Unifies RAG-based prioritization, phishing defense, and explanation verification into a singular agentic pipeline instead of three separate tools, leveraging infrastructure already built for MailPilot.
2. Uses LLM-based semantic reasoning to detect novel/zero-day social-engineering attempts that would otherwise be missed by static rule-based or blacklist approaches.
3. Applies claim-level verification not only for detecting phishing but also for verifying the system's own stated reasoning, plugging a trust gap that no existing agentic AI system (including the author's own previous MailPilot system) currently addresses.
4. Provides an explainable, evidence-linked risk breakdown for both the prioritization and phishing decisions, rather than a binary flag or an unverified natural-language justification.
5. Designed to be self-hostable and lightweight, allowing both phishing defense and verified-explanation AI to be available to individuals, students, and small organizations who cannot afford enterprise security or trust-and-safety tooling.

**4.8 Significance / Industrial Relevance**

Phishing is one of the leading vectors for data breaches and ransomware attacks, and the Anti-Phishing Working Group has repeatedly reported it as one of the most common categories of cybercrime in its quarterly trend reports. At the same time, as agentic AI systems become trusted to act autonomously, flagging, quarantining, or prioritizing content on a user's behalf, the correctness of the explanations such systems give for their actions becomes a safety concern of its own. Small organizations, students, and individual users cannot afford enterprise email-security suites such as Proofpoint or Mimecast, and even fewer tools verify their own explanations at all. A lightweight, self-hostable system that is both a more

capable phishing defense and a demonstrably more trustworthy agentic assistant presents a practical, low-cost protection while showcasing a deployable pattern for building agentic AI systems that check their own reasoning before acting on it.

### 4.9 References

[1] F. Heiding, B. Schneier, A. Vishwanath, J. Bernstein, and P. S. Park, "Devising and Detecting Phishing Emails Using Large Language Models," IEEE Access, vol. 12, pp. 42131-42146, 2024, doi: 10.1109/ACCESS.2024.3375882.
[2] T. Koide, N. Fukushi, H. Nakano, and D. Chiba, "ChatSpamDetector: Leveraging Large Language Models for Effective Phishing Email Detection," arXiv preprint arXiv:2402.18093, 2024.
[3] R. Chataut, P. K. Gyawali, and Y. Usman, "Can AI Keep You Safe? A Study of Large Language Models for Phishing Detection," in Proc. 2024 IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC), Las Vegas, NV, USA, 2024, pp. 548-554.
[4] C. Lee, "Enhancing Phishing Email Identification with Large Language Models," arXiv preprint arXiv:2502.04759, 2025.
[5] S. Asiri, Y. Xiao, S. Alzahrani, and T. Li, "PhishingRTDS: A Real-Time Detection System for Phishing Attacks Using a Deep Learning Model," Computers & Security, vol. 141, p. 103843, 2024, doi: 10.1016/j.cose.2024.103843.
[6] J. Lee, P. Lim, B. Hooi, and D. M. Divakaran, "Multimodal Large Language Models for Phishing Webpage Detection and Identification," in Proc. 2024 APWG Symposium on Electronic Crime Research (eCrime), Boston, MA, USA, 2024, pp. 1-13, doi: 10.1109/eCrime66200.2024.00007.
[7] A. Al-Subaiey, M. Al-Thani, N. A. Alam, K. F. Antora, A. Khandakar, and S. A. U. Zaman, "Novel Interpretable and Robust Web-Based AI Platform for Phishing Email Detection," Computers and Electrical Engineering, vol. 120, p. 109625, 2024, doi: 10.1016/j.compeleceng.2024.109625.
[8] A. Karim, M. Shahroz, K. Mustofa, S. B. Belhaouari, and S. R. K. Joga, "Phishing Detection System Through Hybrid Machine Learning Based on URL," IEEE Access, vol. 11, pp. 36805-36822, 2023, doi: 10.1109/ACCESS.2023.3252366.
[9] K. I. Roumeliotis, N. D. Tselikas, and D. K. Nasiopoulos, "Next-Generation Spam Filtering: Comparative Fine-Tuning of LLMs, NLPs, and CNN Models for Email Spam Classification," Electronics, vol. 13, no. 11, p. 2034, 2024.
[10] J. Hazell, "Spear Phishing With Large Language Models," arXiv preprint arXiv:2305.06972, 2023.
[11] C. Niu et al., "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models," in Proc. 62nd Annual Meeting of the Association for Computational Linguistics, Bangkok, Thailand, 2024, pp. 10862-10878, doi: 10.18653/v1/2024.acl-long.585.
[12] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020, arXiv:2005.11401.
[13] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," in Proc. 2023 Conf. on Empirical Methods in Natural Language Processing (EMNLP), Singapore, 2023, pp. 9004-9017.
[14] S. Min et al., "FActScore: Fine-Grained Atomic Evaluation of Factual Precision in Long Form Text Generation," in Proc. 2023 Conf. on Empirical Methods in Natural Language Processing (EMNLP), 2023, pp. 12076-12100.
[15] S. Dhuliawala et al., "Chain-of-Verification Reduces Hallucination in Large Language Models," arXiv preprint arXiv:2309.11495, 2023.

[16] Y. Tang et al., "MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents," in Proc. 2024 Conf. on Empirical Methods in Natural Language Processing (EMNLP), Miami, FL, 2024, pp. 8818-8847.
[17] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. 2019 Conf. North American Chapter of the Association for Computational Linguistics (NAACL-HLT), 2019, pp. 4171-4186.
[18] Anti-Phishing Working Group (APWG), "Phishing Activity Trends Report," APWG, 2025. [Online]. Available: https://apwg.org/trendsreports/

**Signatures and Approval**

| **Project Guide** | **Project Coordinator** | **Head of Department** |
|---|---|---|
| Signature: _____ | Signature: _____ | Signature: _____ |
| Name: _____ | Name: _____ | Name: _____ |
| Date: _____ | Date: _____ | Date: _____ |